

P1468R2

Fixed-layout floating-point type aliases

Published Proposal, 2019-10-03

This version:

<https://wg21.link/P1468>

Issue Tracking:

[Inline In Spec](#)

Authors:

[Michał Dominiak](#) (NVIDIA)

[David Olsen](#) (NVIDIA)

[Boris Fomitchev](#) (NVIDIA)

[Sergei Nikolaev](#) (NVIDIA)

Audience:

EWG, LEWG

Toggle Diffs:

☐ Hide deleted text

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Table of Contents

1	Abstract
2	Revision history
2.1	R0 -> R1 (pre-Cologne)
2.2	R1 -> R2 (pre-Belfast)
3	Motivation
4	Header name

5 Type aliases

5.1 Supported formats

5.2 Aliasing standard types

5.3 Layout vs. behavior

5.4 Feature test macros

5.5 Names

5.5.1 `floatX_t`

5.5.2 `iec559_binaryX_t`

5.5.3 `binaryX_t`

5.5.4 `fp::binaryX_t`

5.5.5 `fp_binaryX_t`

5.5.6 `iec559::binaryX_t`

6 Literal suffixes

References

Informative References

Issues Index

§ 1. Abstract

This paper proposes a set of `<cstdint>`-style type aliases for floating point types matching specific, well-know floating-point layouts.

This is a companion paper to [\[P1467\]](#), which allows implementations to define floating-point types beyond the three standard types. This paper gives convenient names to some of those types.

§ 2. Revision history

§ 2.1. R0 -> R1 (pre-Cologne)

1. Add the requirement that the types must not alias any of the standard floating-point types.
2. Add a design question about feature-test macros.

3. Add a section on QoI - should we strongly encourage that the aliases to have a hardware implementation?

§ 2.2. R1 -> R2 (pre-Belfast)

Changes based on feedback in Cologne from [SG6](#), [LEWGI](#), and [EWGI](#). Further changes came from further development of the paper by the authors.

- Expanded the [section](#) about whether or not the fixed-layout aliases are allowed to alias standard floating-point types.
- Added a [section](#) about whether the aliases only need to guarantee layout, or should also guarantee behavior.
- Added some text, still preliminary, about [literal suffixes](#).

§ 3. Motivation

16-bit floating-point support is becoming more widely available in both hardware (ARM CPUs and NVIDIA GPUs) and software (OpenGL, CUDA, and LLVM IR). Programmers wanting to take advantage of 16-bit floating-point support have been stymied by the lack of built-in compiler support for the type. A common workaround is to define a class type with all of the conversion operators and overloaded arithmetic operators to make it behave as much as possible like a built-in type. But that approach is cumbersome and incomplete, requiring inline assembly or other compiler-specific magic to generate efficient code.

The problem of efficiently using newer floating-point types that haven't traditionally been supported can't be solved through user-defined libraries. A possible solution of an implementation changing `float` to be a 16-bit type would be unpopular because users want support for newer floating-point types in addition to the standard types, and because users have come to expect `float` and `double` to be 32- and 64-bit types and have lots of existing code written with that assumption.

This problem is worth solving, and there is no viable solution under the current standard. So changing the core language in an extensible and backward-compatible way is appropriate. Providing a standard way for implementations to support 16-bit floating-point types will result in better code, more portable code, and wider use of those types.

[\[P1467\]](#) changes the language so that implementations can support 16-bit and other non-standard floating-point types. This paper gives well-known names to 16-bit and other commonly used floating-point types.

These two papers are the follow-up to [\[P0192\]](#), the `short float` proposal, which was not approved by EWG.

§ 4. Header name

The type aliases proposed here do not fit neatly into any existing header. So we are offering up a strawman proposal of a new header with the name `<fixed_float>`. We are open to other names for the header and to arguments that the type aliases should be added to an existing header.

ISSUE 1 What new or existing header should the type aliases go into?

§ 5. Type aliases

This paper introduces type aliases for several fixed-layout floating-point types. Each alias will be defined only if a type with that layout is supported by the implementation, similar to the `intN_t` aliases.

§ 5.1. Supported formats

We propose aliases for the following layouts:

- [\[IEEE-754-2008\]](#) `binary16` - IEEE 16-bit.
- [\[IEEE-754-2008\]](#) `binary32` - IEEE 32-bit.
- [\[IEEE-754-2008\]](#) `binary64` - IEEE 64-bit.
- [\[IEEE-754-2008\]](#) `binary128` - IEEE 128-bit.
- `bfloat16`, which is `binary32` with 16 bits of precision truncated; see [\[bfloat16\]](#).

`binary32` and `binary64` are the most widely used floating-point types, and are the formats that `float` and `double` have in most implementations. `binary16` is becoming more widely used; see this paper's motivation for details. `binary128` has hardware support in IBM POWER P9 chips. `bfloat16` is used in Google's TPUs and in TensorFlow.

The most widely used format that is not in this list is X87 80-bit. Even though there is hardware support for this format in all current x86 chips, it is used most often because it is the largest type available, not because users specifically want that format.

§ 5.2. Aliasing standard types

ISSUE 2 Can the type aliases proposed in this paper be aliases of standard floating-point types? Or are the type aliases required to alias extended floating-point types?

This has turned out to be the most contentious issue raised in this proposal with strong opinions on both sides. In Cologne, SG6 and LEWGI voted in favor of allowing aliasing of standard types, while EWGI was strongly against the idea. The authors are in favor of prohibiting aliasing of standard types, but realize that not everyone else is convinced of that yet.

The header `<cstdint>` defines integer type aliases for certain integer types, such as `std::int32_t` and `std::int64_t`. These are similar in many ways to the aliases proposed here. The types in `<cstdint>` are allowed to alias standard integer types. That has resulted in compilation errors when users try to create an overload set with both standard types and fixed-layout aliases, such as:

```
int bit_count(int x) { /* ... */ }
int bit_count(std::int32_t x) { /* ... */ }
```

If aliasing of standard types is allowed for the floating-point type aliases, then similar compilation errors will likely result:

```
int get_exponent(double x) { /* ... */ }
int get_exponent(std::float64_t x) { /* ... */ }
```

This is the strongest argument against allowing aliasing of standard types. People who don't find this argument persuasive point out that users should not create overload sets with both standard types and fixed-layout type aliases. An overload set should contain just the standard floating-point types or just the fixed-layout types, but not both. The example above that fails to compile is considered poor design and should not be encouraged.

(The arguments about overload sets apply equally to explicit template specializations.)

Not allowing the aliasing of standard types imposes an implementation burden. If aliasing were allowed, then implementations that don't define any extended floating-point types could define some of the aliases with a little bit of library code that boils down to something like:

```
namespace std {
    using float32_t = float;
    using float64_t = double;
}
```

But when aliasing is not allowed, implementations have to support extended floating-point types in at least the compiler front end, which is not a trivial task. There is also a burden on the name mangling ABI, which will have to define how to encode these extended floating-point types.

The authors feel that the burden on users of allowing aliasing of standard types is greater than the burden on implementers of not allowing such aliasing. Therefore, the authors recommend not allowing aliasing of standard types.

(This argument is predicated on the changes to overload resolution proposed in [\[P1467\]](#). If those changes don't go through, then having `std::float64_t` be an alias of an extended floating-point type rather than an alias of `double` will cause the following code to not compile:

```
void f(std::float32_t);
void f(std::float64_t);
void g(double x) {
    f(x); // error - ambiguous call without overload resolution changes
}
```

If that code doesn't compile, that would be a bigger burden on users than not being able to overload on both `double` and `std::float64_t`. That would change the authors' opinion on the best resolution for this issue.)

§ 5.3. Layout vs. behavior

ISSUE 3 Should the aliases for the IEEE types guarantee that the types have fully conformant IEEE behavior, or only that they have the same layout as the IEEE type?

What type aliases should be defined and how should they behave when the user compiles with the GCC option `-ffast-math`, which turns on optimizations that result in `double` having non-IEEE

behavior in some cases? In that situation, should `std::float64_t` (or whatever it is called) not be defined at all? Or should it be defined and refer to an extended floating-point type that is IEEE conformant? Or should it be defined and refer to a type that, like `double`, is not fully IEEE conformant? Should this be a requirement on the implementation, or just encouragement?

Discussion in SG6 in Kona preferred having the type aliases guarantee full IEEE behavior.

§ 5.4. Feature test macros

Since implementations may choose to support (or not) each of the fixed-layout aliases individually, there should be a separate test macro for detecting each of the type aliases. The names of the test macros would be derived from whichever type alias names we settle on. (The authors are not thrilled with introducing so many new test macros, but they have yet to come up with a better idea.)

ISSUE 4 How should feature test macros be handled for this feature?

§ 5.5. Names

We are proposing several different naming schemes for fixed-layout type alias, and are open to other suggested naming schemes. In committee discussions so far, no set of names has emerged as the favorites. We are leaving it up to the committee to chose.

§ 5.5.1. `floatX_t`

- `std::float16_t`
- `std::float32_t`
- `std::float64_t`
- `std::float128_t`
- `std::bfloat16_t`

This is the simplest of all the options being presented. It is the naming scheme used by Boost.Math's fixed-layout floating-point types.

Nothing in the names of the IEEE aliases implies that they are in fact IEEE binary formats.

Additionally, `float16_t` and `bfloat16_t` are similar enough that we aren't fully comfortable using these names.

§ 5.5.2. `iec559_binaryX_t`

- `std::iec559_binary16_t`
- `std::iec559_binary32_t`
- `std::iec559_binary64_t`
- `std::iec559_binary128_t`
- `std::bfloat16_t`

This naming scheme is as explicit as possible about the layouts that it guarantees. "IEC559" is how the rest of the standard refers to [\[IEEE-754-2008\]](#).

These names are *long*. `iec559` will cause some confusion because is not as recognizable as `ieee754` among non-floating-point experts.

§ 5.5.3. `binaryX_t`

- `std::binary16_t`
- `std::binary32_t`
- `std::binary64_t`
- `std::binary128_t`
- `std::bfloat16_t`

These names are shorter than `std::iec559_binaryX_t`, but they are less obvious; nothing in their names directly points to the being floating-point types.

This is the only option that has received strong SG6 discouragement.

§ 5.5.4. `fp::binaryX_t`

- `std::fp::binary16_t`
- `std::fp::binary32_t`
- `std::fp::binary64_t`
- `std::fp::binary128_t`
- `std::fp::bfloat16_t`

The namespace `fp` makes it more obvious that these types are floating-point types, assisting in the recognition of `binary16` as an [\[IEEE-754-2008\]](#) format. A using namespace directive can be used to avoid repeating `std::fp::` everywhere.

The drawbacks of this approach are that it introduces a new namespace with a very small purpose, and that `std::fp::bfloat16_t` is somewhat redundant with two different floating-point indications (`fp` and the `float` in `bfloat16_t`).

§ 5.5.5. `fp_binaryX_t`

- `std::fp_binary16_t`
- `std::fp_binary32_t`
- `std::fp_binary64_t`
- `std::fp_binary128_t`
- `std::fp_bfloat16_t`

This is a slight modification of the previous scheme, which trades the nested namespace for an `fp_` prefix. The advantages and disadvantages are similar.

§ 5.5.6. `iec559::binaryX_t`

- `std::iec559::binary16_t`
- `std::iec559::binary32_t`
- `std::iec559::binary64_t`
- `std::iec559::binary128_t`
- `std::bfloat16_t` ?? (unsure how to fit `bfloat16` into this scheme)

This scheme was proposed during SG6 discussions in Kona. It is similar to the [namespace `fp`](#) scheme, but with a more precise namespace name.

§ 6. Literal suffixes

Once the names of the aliases have been decided on, a literal suffix for each of those types will be defined, similar to what is proposed in [\[P1280\]](#). Each type will have either two literal operators with `long double` and `unsigned long long` parameters, or (for types whose conversion rank is not

less than `long double`) one literal operator with a `const char *` parameter. The literal operators for an implementation might look like this (with all names subject to change):

```
namespace std {
    inline namespace literals {
        inline namespace fixed_float_literals {
            constexpr float16_t operator""fp16(long double);
            constexpr float16_t operator""fp16(unsigned long long);
            constexpr float32_t operator""fp32(long double);
            constexpr float32_t operator""fp32(unsigned long long);
            constexpr float64_t operator""fp64(long double);
            constexpr float64_t operator""fp64(unsigned long long);
            constexpr float128_t operator""fp128(const char *);
            constexpr bfloat16_t operator""bf16(long double);
            constexpr bfloat16_t operator""bf16(unsigned long long);
        }
    }
}
```

```
constexpr float16_t operator""fp16(long double d);
constexpr float16_t operator""fp16(unsigned long long d);
```

1 *Returns:* float16_t{d}.

```
constexpr float32_t operator""fp32(long double d);
constexpr float32_t operator""fp32(unsigned long long d);
```

2 *Returns:* float32_t{d}.

```
constexpr float64_t operator""fp64(long double d);
constexpr float64_t operator""fp64(unsigned long long d);
```

3 *Returns:* float64_t{d}.

```
constexpr float128_t operator""fp128(const char *s);
```

4 *Effects:* Equivalent to:

```
float128_t x{0};
from_chars(s, s + strlen(s), &x);
return x;
```

```
constexpr bfloat16_t operator""bf16(long double d);
constexpr bfloat16_t operator""bf16(unsigned long long d);
```

5 *Returns:* bfloat16_t{d}.

§ References

§ Informative References

[BFLOAT16]

bfloat16 floating-point format. URL: https://en.wikipedia.org/wiki/Bfloat16_floating-point_format

[IEEE-754-2008]

IEEE Standard for Floating-Point Arithmetic. 29 August 2008. URL: <http://ieeexplore.ieee.org/servlet/opac?punumber=4610933>

[P0192]

Michał Dominiak; et al. [`short float` and fixed-size floating point types](https://wg21.link/P0192). URL: <https://wg21.link/P0192>

[P1280]

Isabella Muerte. [Integer Width Literals](https://wg21.link/P1280). URL: <https://wg21.link/P1280>

[P1467]

Michał Dominiak; David Olsen. [Extended floating-point types](https://wg21.link/P1467). URL: <https://wg21.link/P1467>

§ Issues Index

ISSUE 1 What new or existing header should the type aliases go into? [↩](#)

ISSUE 2 Can the type aliases proposed in this paper be aliases of standard floating-point types? Or are the type aliases required to alias extended floating-point types? [↩](#)

ISSUE 3 Should the aliases for the IEEE types guarantee that the types have fully conformant IEEE behavior, or only that they have the same layout as the IEEE type? [↩](#)

ISSUE 4 How should feature test macros be handled for this feature? [↩](#)